Files, data & exceptions

Essential syntax for text, CSV, JSON, modules, and common errors.

File modes & paths

File modes: **r** = read; **w** = write/replace; **a** = append; **x** = create new.

Relative: "tasks.txt" **Absolute:** "/content/tasks.txt"

with open(...) closes the file when the block finishes.

Write, append & read text

```
with open("notes.txt", "w") as f:
    f.write("Read Python")
with open("notes.txt", "a") as f:
    f.write("\nPractice")
with open("notes.txt", "r") as f:
    print(f.read())
```

Output: Read Python (new line) Practice. Write mode overwrites existing content.

CSV: rows & columns

CSV means Comma-Separated Values. **DictReader** uses the first row as dictionary keys.

```
import csv
with open("marks.csv", newline="") as f:
    for row in csv.DictReader(f):
        print(row["name"], row["marks"])
```

Example headers: name,marks. Values read from CSV are strings unless converted.

JSON: structured records

A **list of dictionaries** stores multiple tasks. **dump()** writes JSON; **load()** reads it.

```
import json
tasks = [{"title": "Read", "done": False}]
with open("tasks.json", "w") as f:
    json.dump(tasks, f, indent=2)
with open("tasks.json", "r") as f:
    saved = json.load(f)
print(saved[0]["title"])
```

Output: Read. **saved[0]** selects the first task; **["title"]** gets its title.

Handling errors: try / except

```
try:
    age = int(input("Enter age: "))
except ValueError:
    print("Enter a whole number")
else:
    print("Valid age:", age)
finally:
    print("Finished")
```

try attempts code; **except** handles an expected error; **else** runs after success; **finally** always runs.

Common errors & import

- **ValueError** - unsuitable input for conversion.
- **FileNotFoundError** - requested file missing.
- **ZeroDivisionError** - dividing by zero.

```
import math  # Load a reusable module
print(math.sqrt(25))  # 5.0
```

Colab reminder: files in the runtime may disappear after it resets.

Functions & object-oriented Python

Build small reusable functions and understand classes, objects, and OOP pillars.

Functions, docs & type hints

A **module** is a Python file with reusable code. A **function** does one clear job.

```
def add_marks(a: int, b: int) -> int:
    """Return the total marks."""
    return a + b
print(add_marks(40, 45))  # 85
```

- **Docstring** describes a function or class.
- **# comment** explains a line of code.
- **Type hints** describe expected types; Python does not automatically enforce them.

Class, object, attributes, methods

Class = blueprint. **Object** = an instance. **Attribute** = stored value. **Method** = function inside a class.

```
class Student:
    def __init__(self, name):
        self.name = name  # Attribute
    def greet(self):   # Method
        return "Hi, " + self.name
ali = Student("Ali")  # Object
print(ali.greet())  # Hi, Ali
```

self refers to the current object; **__init__** sets initial attributes.

Encapsulation

Bundle data and behavior; use methods to validate changes.

```
class Account:
    def __init__(self):
        self._balance = 0
    def deposit(self, amount):
        if amount > 0:
            self._balance += amount
```

_balance means internal by convention; a leading underscore does not enforce privacy.

Inheritance & overriding

Inheritance: a child reuses a parent. **Overriding:** the child defines its own version of a method.

```
class Animal:
    def speak(self):
        return "Sound"
class Dog(Animal):
    def speak(self):
        return "Woof"
print(Dog().speak())  # Woof
```

Polymorphism: different objects can respond differently to the same method name.

Abstraction

Show a simple operation and hide the steps inside it. Callers use the method without managing its internal calculation.

```
class Bill:
    def __init__(self, price, qty):
        self.price = price
        self.qty = qty
    def total(self):
        return self.price * self.qty
print(Bill(100, 3).total())  # 300
```

Composition vs inheritance

- **is-a relationship:** Dog is an Animal - inheritance.
- **has-a relationship:** Library has Books - composition.

```
class Library:
    def __init__(self):
        self.books = []  # Has books
    def add(self, book):
        self.books.append(book)
```

Git, GitHub, HTTP & APIs

Keep a history of code changes and understand how programs exchange data.

Git vs GitHub

Git tracks versions of code. **GitHub** hosts Git repositories online for sharing and collaboration.

- **Repository:** tracked project.
- **Stage:** select changes.
- **Commit:** saved checkpoint with a message.
- **Branch:** separate line of work.

Most useful Git commands

```
git init                # Start repository
git status              # Check changes
git add .               # Stage files
git commit -m "Add task tracker"
git log                 # View commits
git push                # Send to remote
```

README.md explains the project. **.gitignore** excludes chosen files, such as secrets.

Student GitHub submission

- Create repository **mini-task-tracker**.
- Upload **mini_task_tracker.py** and **README.md**.
- Make **at least two meaningful commits**.
- Submit the repository URL to the instructor.

Safety: keep passwords and private records out of your repository.

API, REST & HTTP

API lets one program request data or an action from another. **HTTP** specifies requests/responses between clients and web servers.

- **Client** sends a request.
- **Server** sends a response.
- **REST API** commonly exposes resources using HTTP methods and URLs.

HTTP method & status quick map

GET = read **POST** = create
PUT/PATCH = update **DELETE** = delete

200 OK - success
201 Created - new resource
400 Bad Request - invalid request
404 Not Found - missing resource

Example resource: /tasks. Many APIs exchange JSON dictionaries and lists.

FastAPI, Pydantic, Streamlit

- **FastAPI** - build web endpoints (e.g. GET /tasks).
- **Pydantic** - validate fields and data types.
- **Streamlit** - build a simple Python web interface.

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/tasks")
def get_tasks():
    return [{"title": "Read"}]
```

The slide deck introduces these as next-step tools for later projects.

CRUD, SQLite, testing & project

Apply four data actions, check results, and prepare the Task Tracker submission.

CRUD in Python & SQL

CRUD = Create, Read, Update, Delete.

```
tasks = []
tasks.append("Read")  # Create
print(tasks[0])      # Read
tasks[0] = "Revise" # Update
tasks.remove("Revise")  # Delete
```

SQL mapping: INSERT = Create; SELECT = Read; UPDATE = Update; DELETE = Delete.

SQLite using Python

SQLite stores tables in a local database file. **sqlite3** is Python's built-in module.

```
import sqlite3
db = sqlite3.connect("tasks.db")
db.execute("CREATE TABLE IF NOT EXISTS "
           "tasks (id INTEGER PRIMARY KEY, "
           "title TEXT)")
db.execute("INSERT INTO tasks(title) "
           "VALUES (?)", ("Read",))
db.commit()  # Save changes
db.close()
```

? is a parameter placeholder; pass user data separately instead of building SQL strings.

Testing with assert & pytest

Testing checks actual results against expected results.

```
def is_pass(marks):
    return marks >= 50

def test_pass_mark():
    assert is_pass(50) is True
```

pytest finds functions starting with test_. Run **python -m pytest -v** in a terminal. Check normal, edge, and invalid cases.

Debugging: practical sequence

- 1. **Reproduce** the issue.
- 2. **Read** traceback and line number.
- 3. **Inspect** variables and input values.
- 4. **Fix** one issue and rerun.

```
marks = [80, 90, 100]
print(sum(marks) / len(marks)) # 90.0
```

Mini project: Build a Task Tracker

Task record: {"title": "Read", "done": False}

- **Add:** title + pending state; **View:** numbered tasks.
- **Complete:** set done to True; **Delete:** remove selected task.
- **Save/load:** tasks.json using json.dump/load.
- **Menu:** repeat until Exit; handle invalid entries.

Submission: .py file + README in GitHub; at least two meaningful commits and repository link.

Official reference links

Python Tutorial | JSON | sqlite3

GitHub Docs | pytest | FastAPI